

Bringing QueryGraph AI Navigator Semantics to Apache Polaris

Date: 2026-05-22

Executive Summary

The phrase "semantic layer" is currently carrying two related but different meanings in the lakehouse ecosystem.

The first is the catalog and business semantic layer. Apache Iceberg gives engines precise table semantics through schemas, partition specs, snapshots, metadata tables, delete files, and a REST catalog contract. Unity Catalog and Apache Gravitino extend this idea into governance, federation, tags, lineage, and data and AI asset management. Unity Catalog also now uses "semantic layer" in the analytics sense through metric views: governed definitions of measures, dimensions, filters, and joins. The Apache Polaris OSI proposal fits this first family: it asks whether Polaris should support semantic models, metrics, dimensions, semantic data types, joins, and related APIs.

The second is the machine-readable open data and AI agent semantic layer. QueryGraph.ai, as represented by the local github.com/querygraph reference implementation, describes an AI Navigator bundle that combines Croissant dataset metadata, CDIF/FAIR discovery metadata, DID identity, and ODRL policy in JSON-LD. This is not merely a metrics layer. It is a portable envelope for meaning, provenance, identity, policy, and agent-safe use. The Open Data Spaces initiative points in the same direction: architecture, governance, trust, and ontology-based semantic interoperability across organizations and jurisdictions.

Apache Polaris should support both layers, but with a narrow catalog-first scope. Polaris should not become a metrics compiler, ontology reasoner, policy engine, or AI agent runtime. It should become the governed registration, versioning, discovery, authorization, and lineage control plane for semantic artifacts attached to Iceberg assets. QueryGraph AI Navigator can enter Polaris as a companion envelope for OSI semantic models: OSI defines analytic metrics and dimensions; QueryGraph supplies Croissant, CDIF, DID, and ODRL context so agents, data spaces, and cross-catalog tools know what the model means, who is asserting it, how it can be used, and how it relates to lineage.

The Two Semantic Layer Families

1. Catalog And Business Semantics

This family answers operational questions:

- What is the table, view, namespace, model, or function?
- What schema, partitioning, snapshot, and metadata rules govern it?
- Who can see, alter, or query it?
- What business metrics, dimensions, joins, and calculations should tools use?
- How does the definition move across catalogs and engines?

Apache Iceberg, Unity Catalog, Apache Gravitino, and Apache Polaris mostly live here. So does the Open Semantic Interchange direction described in the Polaris issue.

2. Agent And Open Data Space Semantics

This family answers interoperability and trust questions:

- What does the dataset mean in portable web terms?
- How should an AI agent discover fields, records, files, and intended use?
- Which controlled vocabularies or FAIR metadata describe the asset?
- Which identity asserted the metadata?

- What rights, duties, prohibitions, or constraints travel with the data?
- Which lineage events prove how the semantic artifact was produced or used?

QueryGraph AI Navigator and Open Data Spaces mostly live here. OpenLineage is the bridge that can connect both families through operational events and extensible facets.

Apache Iceberg

Apache Iceberg is not a business semantic layer in the LookML, dbt Semantic Layer, or Unity Catalog metric view sense. Its semantics are table semantics: schema evolution, partition evolution, hidden partitioning, snapshots, time travel, delete files, sort orders, metadata tables, and an atomic commit model. Those features are deeply semantic because they define what a table means over time and how engines can safely read and write it.

The Iceberg REST Catalog API makes those table semantics portable. Engines can load table metadata, refresh cached metadata, commit changes, and obtain access tokens through a catalog service instead of coupling directly to a metastore or object store. Polaris implements this catalog control plane for Iceberg.

Iceberg is therefore the foundation under the semantic layer rather than the whole layer. It can anchor metrics and agent metadata to stable table snapshots, refs, schemas, and metadata locations. That is essential because a metric like `total_revenue` is only reproducible when its physical table, schema, snapshot, and policy context are known.

Unity Catalog

Unity Catalog is a universal governance catalog for data and AI assets. It uses a three-level namespace, catalogs, schemas, and objects such as tables, views, volumes, functions, and models. Its main semantic contribution is a governed asset model: ownership, privileges, discovery metadata, lineage, and policy.

Unity Catalog also exposes a more explicit analytics semantic layer through metric views. Metric views define business measures, dimensions, joins, and filters in a governed catalog object. That places canonical metric definitions near the governed data they summarize. The advantage is consistency: BI tools, SQL users, and agents can ask for the same metric and get the same calculation.

For Polaris, Unity Catalog is the closest practical comparison. It suggests that semantic models should be catalog resources, not loose files. It also suggests that semantic definitions must inherit catalog governance and lineage rather than bypass them.

Apache Gravitino

Apache Gravitino is a federated metadata lake. Its top-level "metalake" spans multiple catalogs and metadata systems. It can model catalogs, schemas, tables, columns, filesets, topics, models, tags, properties, and statistics across heterogeneous backends.

Gravitino's semantic layer is primarily a federation and governance abstraction. It gives organizations a common metadata model over many systems. Tags, properties, statistics, and asset types can carry meaning across domains, but Gravitino is not primarily a business metrics DSL or a JSON-LD agent semantics bundle.

The Polaris lesson is that a semantic API must work across catalog boundaries. Even if Polaris starts with Iceberg, semantic metadata should be exportable to Gravitino-like federated catalogs and importable from them.

OpenLineage

OpenLineage is an open standard for lineage metadata. Its core model describes jobs, runs, datasets, events, and extensible facets. It is not itself a semantic model language, but it is the right event channel for semantic model lifecycle.

Semantic layers need lineage for three reasons:

- Registration lineage: who registered or changed a semantic model?
- Compilation lineage: which physical Iceberg tables, snapshots, and views were used to compile or validate it?
- Query lineage: which semantic model and metric definitions influenced a produced dataset or dashboard?

OpenLineage facets can carry semantic model identifiers, versions, OSI fragments, QueryGraph bundle hashes, ODRL policy identifiers, and DID issuers without forcing the OpenLineage core spec to become a metrics or ontology spec.

QueryGraph.ai And The Local Reference Implementation

The local QueryGraph implementation in `/Users/alexysrc/querygraph/qg-rust` and `/Users/alexysrc/querygraph/qg-python` defines an AI Navigator semantic bundle. The Python implementation builds deterministic JSON-LD that combines four layers:

- Croissant: dataset metadata for machine learning and agent discovery, including records, files, fields, and distribution.
- CDIF: cross-domain FAIR discovery, access, vocabulary, and integration metadata.
- DID: decentralized identity for the agent, service, dataset, or issuer.
- ODRL: machine-readable permissions, prohibitions, duties, and constraints.

This model treats semantics as portable assertions. A bundle can describe not only a metric but also the dataset meaning, rights, attribution, identity, and agent-operable constraints around it. That makes QueryGraph a strong companion to OSI rather than a replacement for OSI.

In a Polaris context, the QueryGraph bundle should be stored as a semantic envelope around a catalog asset:

- The OSI semantic model says "what is `total_revenue`?"
- Iceberg says "what table and snapshot make this reproducible?"
- Polaris says "who can register, read, validate, and use it?"
- QueryGraph says "what does this mean to agents and data spaces, who asserted it, and what usage policy travels with it?"
- OpenLineage says "where did this definition travel and how was it used?"

Open Data Spaces And OSI

The Open Data Spaces initiative from Japan's IPA describes open data space architecture, governance, trust, international data distribution, and ontology and semantic interoperability. Its materials include a reference architecture, governance and trust layers, use case implementation guidance, and an OSI white paper where OSI means Ontology and Semantic Interoperability.

That acronym collides with the Polaris issue's OSI, which means Open Semantic Interchange. The two are compatible but not identical:

- Open Semantic Interchange is a portable analytic semantic model format: metrics, measures, dimensions, joins, calculations, and relationships.

- Ontology and Semantic Interoperability is a broader data space goal: vocabularies, ontologies, mappings, shared meaning, trust, and cross-domain interoperability.

Polaris should avoid using the bare acronym "OSI" in APIs or documentation. Use explicit names such as open-semantic-interchange and ontology-semantic-interoperability where needed.

Open Data Spaces also provides a useful architectural mapping:

- Functional layer: semantic model APIs, catalog discovery, metric resolution.
- Operational layer: OpenLineage events, validation jobs, exchange protocols.
- Governance layer: Polaris privileges, external PDP/OPA, ODRL rights metadata.
- Trust layer: DID issuers, signatures, bundle hashes, provenance attestations.

QueryGraph AI Navigator aligns naturally with this mapping because it already combines data description, identity, and rights metadata in a JSON-LD bundle.

Apache Polaris Issue 4522

The Polaris issue asks whether Polaris should provide first-class support for semantics: semantic models, metrics, and semantic data types. The examples use Open Semantic Interchange-style YAML for metrics and dimensions, including definitions such as:

```
metrics:
  - name: total_revenue
    type: simple
    type_params:
      measure:
        name: revenue

dimensions:
  - name: customer_region
    type: categorical
    expr: customers.region
```

The discussion frames Polaris as a catalog, so the natural question is not "should Polaris become a semantic engine?" but "should Polaris own the governed metadata contract for semantic models?" The strongest answer is yes, with a limited scope:

- Register semantic model artifacts.
- Attach them to catalogs, namespaces, tables, views, branches, tags, and snapshots.
- Version and discover them.
- Authorize access and mutation.
- Validate references to catalog assets.
- Emit lineage.
- Leave compilation, query rewriting, BI serving, ontology reasoning, and agent runtime behavior to external engines.

This aligns with Polaris's Iceberg catalog role and with its external policy decision point integration model.

Comparison Matrix

System	Primary meaning of "semantic"	Best contribution to Polaris	Boundary to preserve
Apache Iceberg	Table correctness over time: schema, partitioning, snapshots, commits	Stable physical anchor for semantic models	Do not turn Iceberg table metadata into a business ontology
Apache Polaris	Catalog control plane for Iceberg assets	Governed registration, auth, versioning, discovery, lineage	Do not become a metrics compiler or AI runtime
Unity Catalog	Governed data and AI assets, plus metric views	First-class semantic objects with permissions and lineage	Avoid copying a vendor-specific metric model
Apache Gravitino	Federated metadata model across systems	Cross-catalog asset mapping, tags, properties, federation	Keep Polaris APIs portable and Iceberg-native
OpenLineage	Job, run, dataset lineage events with facets	Lifecycle and usage events for semantic artifacts	Do not overload lineage events as the source of truth
QueryGraph.ai	JSON-LD agent metadata: Croissant, CDIF, DID, ODRL	AI Navigator envelope for meaning, policy, identity, FAIR data	Do not replace OSI metric definitions
Open Data Spaces	Governance, trust, and ontology-based interoperability	Architecture lens for data space conformance	Avoid acronym confusion with Open Semantic Interchange

Proposed Polaris Integration

1. Add A Semantic Model Catalog Resource

Introduce a new Polaris resource type:

`SemanticModel`

A semantic model belongs to a catalog and namespace, and may target a table, view, namespace, catalog, or external asset. The minimal metadata should include:

- `name`
- `format`
- `formatVersion`
- `target`
- `semanticModel`
- `artifactUri`
- `artifactHash`
- `querygraphBundle`
- `lineageFacets`
- `createdBy`
- `createdAt`
- `modifiedAt`
- `snapshotRef` or `tableSnapshotId` when bound to Iceberg state

The `semanticModel` field can hold an Open Semantic Interchange document or a reference to one. The `querygraphBundle` field can hold either inline JSON-LD for small examples or a URI and content hash for production use.

2. Use Explicit Formats

Avoid one universal semantic DSL. Support explicit format names:

```
open-semantic-interchange
querygraph-ai-navigator-jsonld
mlcommons-croissant
codata-cdif
w3c-odrl
w3c-did
```

Polaris validates the envelope, references, identity, and permissions. External compilers validate deeper metric or ontology semantics.

3. Attach Semantics To Iceberg State

Semantic models should optionally bind to:

- Current table metadata.
- A specific snapshot ID.
- A branch or tag.
- A schema ID.
- A view definition version.

This allows reproducible metric interpretation. For example, `total_revenue` registered against orders at snapshot 12345 can remain interpretable even after `orders.order_total` is renamed or retyped.

4. Align Authorization With Polaris And ODRL

Polaris should enforce local privileges such as:

- `SEMANTIC_MODEL_CREATE`
- `SEMANTIC_MODEL_READ`
- `SEMANTIC_MODEL_ALTER`
- `SEMANTIC_MODEL_DROP`
- `SEMANTIC_MODEL_VALIDATE`
- `SEMANTIC_MODEL_USE`

ODRL should travel as rights metadata inside the `QueryGraph` bundle, but Polaris should not treat ODRL as the only enforcement engine. Instead:

- Polaris performs standard privilege checks.
- Polaris can call its external PDP/OPA integration.
- The PDP can inspect ODRL policies when useful.
- Downstream data space participants can read the ODRL policy as portable rights metadata.

5. Emit OpenLineage Events

Polaris or its sidecar service should emit OpenLineage events when:

- A semantic model is registered.
- A semantic model is validated or compiled.
- A query uses a semantic model.
- A model is exported to a data space or another catalog.

Use a custom facet such as:

```
{
  "querygraph_semantic_model": {
    "_producer": "https://querygraph.ai",
    "_schemaURL": "https://querygraph.ai/schemas/openlineage/semantic-model-facet.json",
    "semanticModel": "revenue_semantics",
    "format": "open-semantic-interchange",
    "querygraphBundleHash": "sha256:..."
  }
}
```

6. Provide A Sidecar Before Core API Adoption

The least disruptive path is a sidecar integration:

1. Store semantic artifacts in an object store or repository.
2. Register references and hashes as Polaris properties or generic entities.
3. Validate Iceberg table references through Polaris.
4. Emit OpenLineage events.
5. Use OPA to gate create, alter, read, and use actions.

After proving the contract, Polaris can promote this into a first-class API.

Proposed API Sketch

```
POST /api/catalogs/{catalog}/namespaces/{namespace}/semantic-models
GET  /api/catalogs/{catalog}/namespaces/{namespace}/semantic-models/{name}
PUT  /api/catalogs/{catalog}/namespaces/{namespace}/semantic-models/{name}
POST /api/catalogs/{catalog}/namespaces/{namespace}/semantic-models/{name}:validate
POST /api/catalogs/{catalog}/namespaces/{namespace}/semantic-models/{name}:compile
DELETE /api/catalogs/{catalog}/namespaces/{namespace}/semantic-models/{name}
```

Example payload:

```
{
  "name": "revenue_semantics",
  "format": "open-semantic-interchange",
  "formatVersion": "0.1",
  "target": {
    "type": "iceberg-table",
    "identifier": "lakehouse.analytics.orders",
    "snapshotId": "12345"
  },
  "semanticModel": {
    "metrics": [
      {
        "name": "total_revenue",
        "type": "simple",
        "type_params": {
          "measure": {
            "name": "revenue"
          }
        }
      }
    ]
  }
}
```

```

    "format": "querygraph-ai-navigator-jsonld",
    "artifactUri": "s3://semantic/revenue_semantics/querygraph.jsonld",
    "artifactHash": "sha256:..."
  }
}

```

How QueryGraph AI Navigator Fits

QueryGraph should be introduced as an enrichment and governance envelope:

1. Convert an OSI semantic model into or attach it to a QueryGraph AI Navigator bundle.
2. Use Croissant to describe dataset fields, record sets, files, examples, and ML usability.
3. Use CDIF to add FAIR discovery, access, and vocabulary metadata.
4. Use DID to identify the issuer, agent, service, or organization making the semantic assertion.
5. Use ODRL to carry rights and constraints across catalogs and data spaces.
6. Register the resulting model and bundle reference in Polaris.
7. Emit OpenLineage facets for registration, validation, and use.

This gives Polaris an AI-ready semantic story without making Polaris an AI runtime.

Alignment With Open Data Spaces

The integration aligns with Open Data Spaces when implemented as follows:

- Architecture: Polaris is the catalog and governance control plane. QueryGraph is the semantic envelope. OpenLineage is the operational trace. Iceberg is the table substrate.
- Governance by design: Polaris privileges and OPA policies control who can create and use semantic models. ODRL policies travel with the artifact.
- Trust by design: DID issuers, artifact hashes, and optional signatures make semantic assertions verifiable.
- Ontology and semantic interoperability: OSI metrics can link to CDIF vocabulary terms, schema.org/Croissant fields, DCAT distributions, and domain ontologies.
- International data distribution: The bundle is JSON-LD and can be exchanged with data spaces without requiring all participants to use Polaris.

Implementation Roadmap

Phase 0: Sidecar Reference

Use this repository as a minimal sidecar:

- Build a QueryGraph AI Navigator bundle from a model.
- Produce a Polaris registration payload.
- Produce an OpenLineage event.
- Evaluate OPA policy input.

Phase 1: Polaris Extension

Add a generic semantic artifact registration service beside Polaris:

- Persist semantic model documents and QueryGraph bundles.
- Store references, hashes, and target identifiers in Polaris properties or a parallel metadata store.
- Validate Iceberg identifiers through the Polaris catalog API.
- Emit OpenLineage.

Phase 2: First-Class Polaris API

Promote the sidecar contract into Polaris:

- Add SemanticModel as a catalog entity.
- Add privileges and audit events.
- Add validation hooks for target tables, views, and snapshots.
- Add import/export for Open Semantic Interchange documents.

Phase 3: Open Data Spaces Conformance

Layer in trust and interoperability:

- DID issuer verification.
- Bundle hash and signature verification.
- ODRL-to-PDP policy mapping.
- CDIF and ontology mapping validation.
- OpenLineage conformance tests.

Risks And Design Guardrails

- Do not invent a new metric DSL. Use Open Semantic Interchange or another explicit external format.
- Do not make OpenLineage the source of truth. It is the event trail.
- Do not use ODRL as a substitute for Polaris authorization. ODRL is portable rights metadata and policy input.
- Do not inline large JSON-LD bundles in catalog rows. Store by URI and hash in production.
- Do not ignore Iceberg versions. Bind semantic models to snapshots, schema IDs, or refs where reproducibility matters.
- Do not let the OSI acronym become ambiguous. Spell out the intended standard.

Recommendation

Apache Polaris should accept the spirit of issue 4522 and add first-class semantic model metadata, but it should keep the API catalog-centered. The best design is not "Polaris as a semantic engine"; it is "Polaris as the governed semantic registry for Iceberg."

QueryGraph AI Navigator should be brought in as the AI and open data space envelope around Open Semantic Interchange models. That creates a layered architecture:

Open Semantic Interchange: metrics, dimensions, joins, measures

QueryGraph AI Navigator: Croissant, CDIF, DID, ODRL JSON-LD envelope

OpenLineage: lifecycle and usage events

Apache Polaris: registration, authorization, versioning, discovery

Apache Iceberg: reproducible table and snapshot substrate

Open Data Spaces: architecture, governance, ontology interoperability, trust

This is aligned with Polaris's role as a catalog, Unity Catalog's evidence that semantic models belong near governance, Gravitino's federation direction, OpenLineage's event model, and Open Data Spaces' focus on trustable interoperability.

Source Links

- Apache Polaris issue 4522: <https://github.com/apache/polaris/issues/4522>
- Apache Polaris documentation: <https://polaris.apache.org/releases/1.5.0/>
- Apache Polaris external PDP/OPA documentation: <https://polaris.apache.org/releases/1.5.0/managing-security/external-pdp/opa/>
- Apache Iceberg documentation: <https://iceberg.apache.org/docs/latest/>
- Apache Iceberg REST Catalog API: <https://iceberg.apache.org/rest-catalog-spec/>
- Unity Catalog documentation: <https://docs.unitycatalog.io/>
- Databricks metric views documentation: <https://docs.databricks.com/aws/en/metric-views>

- Apache Gravitino documentation: <https://gravitino.apache.org/docs/next/>
- OpenLineage object model: <https://openlineage.io/docs/spec/object-model/>
- MLCommons Croissant specification: <https://docs.mlcommons.org/croissant/docs/croissant-spec-1.1.html>
- CODATA Cross-Domain Interoperability Framework: <https://codata.org/initiatives/making-data-work/cdif/>
- W3C DID Core: <https://www.w3.org/TR/did-1.0/>
- W3C ODRL Information Model: <https://www.w3.org/TR/odrl-model/>
- Open Data Spaces initiative: <https://www.ipa.go.jp/en/digital/opendataspaces/>
- QueryGraph site: <https://querygraph.ai/>
- Local QueryGraph reference implementation: `/Users/alexy/src/querygraph/qg-rust` and `/Users/alexy/src/querygraph/qg-python`